

5 Projetos Práticos com Python e MongoDB Atlas

Aplicações NoSQL Modernas: GeoJSON, TTL, Time Series e Vector Search

Visão Geral

Este documento detalha cinco propostas de projetos integrando o ecossistema **Python** (`pymongo`, `FastAPI`, `motor`) aos recursos avançados do **MongoDB Atlas**, destacando as capacidades que diferenciam o modelo NoSQL orientado a documentos.

1. Radar de Preços e Notificador de Quedas

Stack Tecnológica: Python (`requests`, `BeautifulSoup` / `Playwright`, `pymongo`), `MongoDB Atlas` e bot de mensageria (`python-telegram-bot` ou e-mail).

Diferenciais no MongoDB Atlas:

- **Subdocumentos para Histórico:** Armazenamento da série de preços dentro do documento do produto:

```
historico_precos = [{data: ISODate, preco: 149.90}, ...]
```

- **Aggregation Pipelines:** Uso de estágios (`$unwind`, `$group`, `$min`, `$avg`) para calcular em tempo real métricas como menor valor histórico e média móvel de 30 dias.

Aplicação Prática: Monitoramento contínuo de oscilações de preço em e-commerces, evitando falsas promoções e alertando usuários quando metas forem atingidas.

2. Guardiã de Sessões e Links Temporários (Expiring Vault)

Stack Tecnológica: Python (`FastAPI`, `pydantic`, `cryptography`, driver assíncrono `motor`).

Diferenciais no MongoDB Atlas:

- **Índices TTL (*Time-To-Live*):** Criação de índice automático no campo de data de expiração (`expireAt`). O mecanismo interno do MongoDB remove os documentos expirados sem necessidade de rotinas periódicas (*Cron Jobs*).
- **Esquema Flexível Criptografado:** Suporte nativo ao armazenamento de cargas BSON encriptadas com controle de visualização única (*burn-after-reading*).

Aplicação Prática: Compartilhamento seguro de credenciais provisórias em ambientes corporativos e mensagens que expiram após a leitura.

3. Rastreador de Eventos Locais e Pontos de Interesse (GeoEvents)

Stack Tecnológica: Python (FastAPI / Flask, pymongo), integração com mapas (*Leaflet.js* ou *Mapbox*).

Diferenciais no MongoDB Atlas:

- **Índice Geoespacial 2dsphere:** Permite indexação de coordenadas no padrão GeoJSON do tipo Point ou Polygon.
- **Consultas por Raio de Proximidade:** Uso dos operadores `$near` e `$maxDistance` para filtrar eventos a uma distância máxima d da localização do usuário:

$$d \leq r_{\max} \quad (\text{em metros na esfera terrestre})$$

Aplicação Prática: Plataformas urbanas de turismo, delivery e localização de meetups e eventos culturais em tempo real.

4. Hub de Telemetria IoT e Análise Energética

Stack Tecnológica: Python (paho-mqtt, FastAPI, pandas, Streamlit para visualização de métricas).

Diferenciais no MongoDB Atlas:

- **Time Series Collections:** Coleções nativas otimizadas para ingestão de séries temporais com compressão colunar interna de alta performance.
- **Operadores \$densify e \$bucket:** Preenchimento automatizado de lacunas na telemetria e agrupamento de leituras (potência em kW, corrente em A) em janelas temporais fixas.

Aplicação Prática: Monitoramento de eficiência energética em data centers, residências inteligentes e estufas agrícolas.

5. Recomendador Semântico de Artigos com Atlas Vector Search

Stack Tecnológica: Python (sentence-transformers / OpenAI Embeddings, LangChain, pymongo).

Diferenciais no MongoDB Atlas:

- **Atlas Vector Search:** Armazena vetores densos $v \in \mathbb{R}^d$ no próprio documento NoSQL.
- **Similaridade de Cosseno:** Busca vetorial com o operador `$vectorSearch`, calculando a relevância semântica:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Aplicação Prática: Motores de recomendação baseados em contexto, busca por intenção em documentações e arquiteturas RAG.

Snippet Base de Conexão com MongoDB Atlas

```
import os
from pymongo import MongoClient
from pymongo.server_api import ServerApi

# Obtenha a URI de conexão a partir de variáveis de ambiente
MONGO_URI = os.getenv("MONGO_URI", "mongodb+srv://<user>:<password>@cluster.
    mongodb.net/?retryWrites=true&w=majority")

client = MongoClient(MONGO_URI, server_api=ServerApi('1'))

try:
    client.admin.command('ping')
    print("Conexao estabelecida com sucesso com o MongoDB Atlas!")
    db = client["projetos_python"]
    colecao = db["registros"]
except Exception as e:
    print(f"Erro ao conectar: {e}")
```